Hardware-Aware Correctness Benchmarking for LLM-Generated Quantum Code

Harmandeep Kaur
School of Computing
University of Connecticut
Storrs, CT, USA
harman.jld@gmail.com

***Abstract*—Large language models are increasingly used to generate quantum programs, yet their outputs are predominantly evaluated for functional correctness alone, namely whether a generated circuit reproduces the expected statevector or expectation value under noiseless simulation. This criterion is necessary but insufficient: a circuit that is functionally correct in simulation may be unexecutable or severely degraded on real quantum hardware due to native gate-set mismatches, qubit-connectivity violations, depth and gate-count inflation introduced during transpilation, and sensitivity to device noise. We argue that this simulation-to-hardware gap is a consequential failure mode of LLM-based quantum code generation that has not been measured systematically. We introduce a benchmark that evaluates generated quantum code along two distinct axes: functional correctness and hardware executability. The hardware-executability axis captures native-gate compliance, connectivity feasibility, post-transpilation depth and two-qubit gate overhead, and an estimated success probability on representative NISQ backends, computed analytically from published calibration data rather than measured on a device. Using a task suite spanning common circuit-construction and algorithmic prompts, we evaluate several state-of-the-art LLMs and quantify how often functionally correct generations fail to maintain a high estimated success probability or competitive efficiency once mapped to a target device model. Our results expose systematic, model-dependent weaknesses that simulation-only evaluation conceals, and provide a reproducible methodology and metric set for hardware-aware assessment of AI-assisted quantum programming. We release the benchmark to support standardized, execution-grounded evaluation of future Q-GenAI systems.**

***Index Terms*—quantum code generation, large language models, hardware-aware benchmarking, LLM code evaluation, quantum circuit transpilation, NISQ hardware constraints, semantic correctness, reproducible benchmarking, AI-assisted quantum programming, quantum software engineering**

I. Introduction

Large language models (LLMs) have become practical assistants for quantum software development. They draft Qiskit programs, construct parameterized ansätze, and translate algorithmic descriptions into executable circuits [4]. A growing body of work evaluates this capability using functional correctness: the generated code is compiled, run on a noiseless simulator, and its output statevector, unitary, or expectation value is compared against a reference [3].

Functional correctness is a necessary condition, but it is not the condition under which quantum programs actually run. Real devices impose a native gate set, restricted qubit connectivity, and gate errors that accumulate with circuit depth and two-qubit gate count. A generated circuit that is exactly correct in simulation may, after transpilation to a target device, inflate in depth and gate count to the point where decoherence dominates the computation. In the NISQ regime [1], these are not implementation details; they determine whether a program produces usable output at all.

We refer to the discrepancy between simulator-validated behavior and device-mapped behavior as the simulation-to-hardware gap. It is a distinct failure mode of LLM-generated quantum code, and it has not been measured systematically. Existing quantum code benchmarks stop at noiseless simulation, so two models that appear equivalent under simulation-only scoring can differ substantially in the hardware viability of their outputs.

This paper makes three contributions. First, a two-axis evaluation framework that separates functional correctness (does the circuit compute the right thing under ideal simulation) from hardware executability (can the circuit be mapped to a representative NISQ device with acceptable overhead and predicted success probability), with concrete, reproducible metrics for each, including a threshold-parameterized hardware-conditioned pass rate and a relative-efficiency metric that normalizes against expert reference solutions. Second, a benchmark suite of 16 circuit-construction and algorithmic prompts with machine-checkable references, evaluated against two backend models spanning the topology and gate-basis extremes of current NISQ platforms. Third, an empirical study of five state-of-the-art LLMs (399 generations). At an estimated-success-probability threshold of $\tau = 0.9$ on a trapped-ion-style target, the four frontier models lose between 26 and 39 percentage points of their functional pass rate, a mean simulation-to-hardware gap of 33.6 points that is invisible to simulation-only scoring. Among the circuits that are functionally correct, 39 to 46 percent fall below the threshold, and this rate is similar for all four models. We also find that the gap depends on the threshold and the target. On a calibrated heavy-hex superconducting model, circuits at benchmark scale saturate the binary metric until $\tau \geq 0.95$, and the differences between models show up instead in the continuous success-probability and structural metrics.

The benchmark, reference solutions, evaluation harness, and all model generations are released at https://github.com/

harmanJLD/hardware-aware-qcode-bench.

II. Related Work

LLMs for quantum code. General-purpose and fine-tuned LLMs are applied to quantum program synthesis and assistant tooling, most prominently the Qiskit Code Assistant [4], and are typically validated by unit tests or simulator equivalence.

Quantum code benchmarks. Qiskit HumanEval [3] adapts the pass@k paradigm from classical code evaluation [2] to quantum programs, scoring compilation success and simulated output equivalence. Application-level suites such as QASM-Bench [5] and SupermarQ [6] characterize fixed circuits for benchmarking devices. Our work connects the two lines: we apply hardware-characterization metrics to the outputs of generative models rather than to a fixed corpus.

Transpilation and noise-aware compilation. Qubit routing, basis decomposition, and noise-adaptive mapping are well studied [7]–[9]. We do not propose new compilation techniques; we use standard transpilation as the measurement instrument that exposes generator weaknesses.

III. Benchmark Design

A. Task Suite

The suite contains 16 prompts in three categories, each a natural-language specification with explicit qubit counts and a machine-checkable reference: state preparation (6 tasks: Bell, GHZ-4, GHZ-8, W-3, uniform superposition, basis-state encoding; reference is the target statevector up to global phase), canonical algorithms (6 tasks: 4-qubit QFT checked by unitary equivalence, one Grover iteration, phase-oracle Bernstein-Vazirani and Deutsch-Jozsa, textbook phase estimation of the T gate, and the SWAP test; references are statevectors or unitaries), and variational workloads (4 tasks: a fully specified hardware-efficient ansatz, a depth-1 QAOA MaxCut circuit on the 4-ring checked by cost-observable expectation, a first-order Trotter step, and a two-local circular-entanglement ansatz). Statevector checks require state fidelity ≥ 0.99; expectation checks use tolerance 10^{-4}.

B. Axis 1: Functional Correctness

Generated code executes in a sandbox and must define a Qiskit circuit. We record the compile rate C (parses and constructs a circuit without runtime error) and the functional pass rate F (satisfies the task's reference check under noiseless simulation). We sample $k = 5$ generations per task per model at temperature 0.8 and report mean pass rate and pass@5 [2].

C. Axis 2: Hardware Executability

Each generation is transpiled to each target at optimization level 3 with fixed seeds (SABRE routing [7]). We compute: native-gate compliance G, the fraction of pre-transpilation gates already in the target basis; connectivity violations V, two-qubit interactions infeasible under the trivial layout; depth inflation D and two-qubit overhead T, the post- to pre-transpilation ratios; and an estimated success probability $\hat{F}$, the product of $(1-\epsilon_i)$ over post-transpilation operations using the backend's reported error rates. We use *estimated* literally: $\hat{F}$ is an analytic quantity derived from calibration data, not a measured output fidelity or a noisy simulation, and it should be read as a relative proxy for device viability rather than a predicted device result. The hardware-conditioned pass rate $F_H(\tau)$ is the fraction of generations that are functionally correct and achieve $\hat{F} \geq \tau$ on target H; the simulation-to-hardware gap is $\Delta_H(\tau) = F - F_H(\tau)$. Because Δ_H conflates generation inefficiency with intrinsic task cost, we also report relative efficiency $\hat{F}_{\text{gen}}/\hat{F}_{\text{ref}}$ against an expert reference solution transpiled identically, which isolates the success probability sacrificed by the generator.

TABLE I

Main results. C: compile rate. F: functional pass rate (noiseless), with 95% bootstrap CI. $\bar{F}_{\text{ion}}$: mean estimated success probability of correct circuits on the ion-style target. Δ_{ion}: gap at $\tau = 0.9$. Displayed values are rounded; means and gaps are computed from unrounded quantities.

Model	C	F	p@5	$\bar{F}_{\text{ion}}$	$F_{\text{ion}}(0.9)$	Δ_{ion}
Gemini 2.5 Pro	.94	.90 [.82,.96]	1.00	.83	.51	.39
Claude Sonnet 4.5	1.00	.83 [.74,.90]	.94	.84	.46	.36
GPT-4o	.91	.71 [.61,.81]	.88	.82	.39	.33
DeepSeek-V3	.89	.68 [.58,.78]	.75	.83	.41	.26
Qwen2.5-C-32B	.19	.19 [.10,.28]	.19	.72	.13	.06

D. Target Backends

We evaluate two backend models chosen to span current NISQ extremes: a 133-qubit IBM Heron heavy-hex model (`FakeTorino`: sparse connectivity, {CZ, RZ, SX, X} basis, published calibration data) and a synthetic 25-qubit trapped-ion-style model (all-to-all connectivity, {RXX, RZ, RX, RY} basis, uniform seeded error rates via Qiskit's generic backend). Using device models rather than live hardware keeps the benchmark reproducible and free to run; the ion-style target should be read as a topology and basis contrast, not as a calibrated vendor device.

IV. Experimental Setup

We evaluate five models accessed through a unified API gateway on July 10, 2026: Gemini 2.5 Pro, Claude Sonnet 4.5, GPT-4o, DeepSeek-V3, and Qwen2.5-Coder-32B-Instruct. Each model receives identical prompts and a system instruction to return only Qiskit code; generations get one attempt with no repair loop, isolating single-shot capability. The harness (Python, Qiskit 2.x) automates sandboxed execution, checking, transpilation, and metrics; 399 of 400 generations completed (one Gemini sample was lost to an API transport error). We report bootstrap 95% confidence intervals over generations.

V. Results

A. Simulation-Only Scoring Overstates Capability

Table I summarizes the main results. At $\tau = 0.9$ on the ion-style target, the four frontier models lose 26 to 39 percentage points relative to their functional pass rates (mean 33.6 points).

TABLE II

STRUCTURAL HARDWARE METRICS OVER FUNCTIONALLY CORRECT GENERATIONS. G: NATIVE-GATE COMPLIANCE (HEAVY-HEX BASIS). V: MEAN CONNECTIVITY VIOLATIONS. D, T: MEDIAN DEPTH INFLATION AND TWO-QUBIT OVERHEAD AFTER TRANSPILATION TO HEAVY-HEX. RE: MEAN RELATIVE EFFICIENCY ON THE ION-STYLE TARGET; %<.9: FRACTION OF CORRECT CIRCUITS BELOW 0.9 RELATIVE EFFICIENCY.

Model	G	V	D	T	RE	%<.9
Gemini 2.5 Pro	.13	.79	3.1	1.8	.93	8
Claude Sonnet 4.5	.14	.55	3.1	1.8	.93	8
GPT-4o	.12	.35	3.1	1.0	.92	9
DeepSeek-V3	.09	.39	3.0	1.0	.91	11
Qwen2.5-C-32B	.33	2.00	3.0	2.7	.73	33

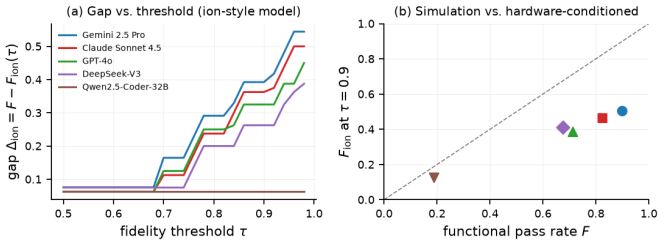


Fig. 1. (a) The simulation-to-hardware gap $\Delta_{\text{ion}}(\tau)$ grows with the threshold and is largest for the most capable models. (b) All models fall below the diagonal: functional pass rate systematically overstates hardware-conditioned pass rate at $\tau = 0.9$.

Among the circuits that are functionally correct, between 39% (DeepSeek-V3) and 46% (GPT-4o) fall below the estimated-success-probability floor, so the conditional hardware-failure rate is similar for all four frontier models. This has a direct consequence for rankings. A simulation-only leaderboard reports that Gemini 2.5 Pro achieves a 90% functional pass rate and says nothing about the fact that it also has the largest hardware gap, 39 points. Figure 1(a) shows that this ordering of gaps holds across the whole threshold range. Because the raw gap Δ_H is bounded above by the functional pass rate, it partly rewards weak generators; the conditional failure rate among correct circuits and the relative efficiency of Table II are the fairer quantities for comparing models, and we read the rankings from those.

B. The Gap Is Threshold- and Target-Dependent

On the calibrated Heron heavy-hex model, every functionally correct generation from every model clears $\tau = 0.9$, and mean relative efficiency is indistinguishable from 1.0. Calibration on current superconducting hardware, combined with optimization-level-3 transpilation, absorbs the inefficiency of LLM output at these circuit sizes, so the binary metric saturates and only begins to separate models at $\tau \geq 0.95$. The methodological lesson is that hardware-conditioned pass rates should be reported as threshold curves rather than single numbers, and that continuous metrics (success-probability estimates, relative efficiency, structural overheads) do the discriminating where the binary metric cannot. The contrast between the two targets also shows that, at these circuit widths, the gap comes mainly from gate-basis decomposition cost and per-gate error rates rather than from connectivity. Connectivity violations average below 0.8 per correct frontier-model circuit (Table II).

C. Structural Failure Modes

Table II decomposes the sources of overhead. All models write idealized textbook circuits. Native-gate compliance is 9 to 14% for the frontier models, so nearly every gate has to be decomposed, and the median correct circuit grows 3.0 to 3.1x in depth when mapped to heavy-hex. Two-qubit overhead separates models more sharply: GPT-4o and DeepSeek-V3 emit circuits whose two-qubit count survives transpilation unchanged (median 1.0), while Gemini and Claude incur 1.8x and Qwen2.5-Coder 2.7x. Relative efficiency shows that when the frontier models are correct, they give up 7 to 9% of the estimated success probability that an expert reference achieves on the ion-style target. The distribution has a heavy tail, with 8 to 11% of correct circuits falling below 0.9 relative efficiency. For Qwen2.5-Coder the mean loss is 27%, and a third of its correct circuits fall below 0.9.

D. A Case Study in What pass@k Conflates

Qwen2.5-Coder-32B fails to compile 81% of its generations. Three quarters of all its samples are Python syntax errors, which points to output-formatting and instruction-following failures rather than errors in quantum reasoning. All 15 of its generations that did compile were functionally correct. A simulation-only pass@k score of 0.19 suggests the model cannot write quantum programs, but the decomposition shows a formatting-reliability problem sitting in front of intact circuit-construction ability, with poor hardware efficiency (Table II) as a separate third weakness. A benchmark that reports a single scalar mixes all three together.

E. Task Difficulty Tracks Convention Sensitivity

Pooled across models, the hardest tasks are those whose specifications pin conventions: the W state (12% pass), phase-oracle Deutsch-Jozsa (20%), phase estimation of the T gate (28%), and QFT with explicit swap and endianness requirements (40%). The models often produce a valid circuit for a different convention (opposite endianness, missing swaps, wrong oracle sign), while the unconstrained state-preparation tasks come close to ceiling. For AI-assisted quantum programming this suggests that following the specification, rather than knowing the gates, is what limits functional correctness.

VI. DISCUSSION AND LIMITATIONS

Our results indicate that simulation-only scoring overstates the practical capability of LLM quantum programmers, and that the overstatement is model-dependent and largest for the models that top simulation-only rankings. Hardware-aware metrics are cheap to add to any existing benchmark, since they require only transpilation and calibration data rather than device access.

Limitations: (i) $\hat{F}$ is an analytic estimate from calibration snapshots, not a device run or a noisy simulation, and the ion-style target uses synthetic uniform error rates, so cross-target comparisons reflect error-rate assumptions as well as topology and basis; (ii) the task suite emphasizes small and mid-sized circuits where exact reference checking is tractable, which is the same regime where the heavy-hex metric saturates, and larger tasks would likely open the gap on superconducting targets as well; (iii) we evaluate single-shot generation, whereas deployed assistants iterate against compiler feedback, and whether repair loops close the hardware gap or only the compile gap is a natural next experiment; (iv) convention-sensitive tasks conflate specification-following with circuit knowledge by design, and disentangling the two further would require paired prompts; (v) the bootstrap resamples generations without modelling the clustering of the $k = 5$ samples drawn per task, so the intervals in Table I are narrower than a task-clustered estimate would give; (vi) relative efficiency is normalised against one expert reference per task, which is a strong but not provably optimal circuit, so RE measures success probability sacrificed against that baseline rather than against the true optimum; (vii) the functional checks compare final states, unitaries, or single expectation values, so a circuit that reaches the reference output by an unintended route is scored correct; and (viii) prompts specify the task only, not the target basis, coupling map, or an optimization objective, so the benchmark measures how well generic Qiskit output survives compilation rather than whether a model can be directed to produce hardware-aware code, and target-conditioned prompting is a natural follow-up.

VII. Conclusion

We introduced a benchmark that evaluates LLM-generated quantum code along functional correctness and hardware executability, defined a reproducible metric set including threshold-parameterized hardware-conditioned pass rates and reference-normalized relative efficiency, and measured a substantial, model-dependent simulation-to-hardware gap across five state-of-the-art LLMs: a mean of 33.6 percentage points at $\tau = 0.9$ on a trapped-ion-style target, with 39 to 46% of functionally correct frontier-model circuits falling below the estimated-success-probability floor. We release the benchmark, harness, and all generations to support execution-grounded evaluation of future Q-GenAI systems, and we encourage the community to report hardware-conditioned pass rates alongside pass@k.

Acknowledgments

The author thanks the Department of Computer Science and Engineering at the University of Connecticut for its support of this work. During the preparation of this work, the author used Claude (Anthropic) for code verification and manuscript drafting/grammar assistance. All experiments were executed and all reported numbers verified by the author, who reviewed and edited the content and takes full responsibility for the publication's content. The model generations analysed in this paper are experimental data, not authoring assistance.

References

[1] J. Preskill, "Quantum computing in the NISQ era and beyond," *Quantum*, vol. 2, p. 79, 2018.
[2] M. Chen *et al.*, "Evaluating large language models trained on code," arXiv:2107.03374, 2021.
[3] S. Vishwakarma *et al.*, "Qiskit HumanEval: An evaluation benchmark for quantum code generative models," in *Proc. IEEE Int. Conf. Quantum Comput. Eng. (QCE)*, 2024.
[4] N. Dupuis *et al.*, "Qiskit Code Assistant: Training LLMs for generating quantum computing code," arXiv:2405.19495, 2024.
[5] A. Li, S. Stein, S. Krishnamoorthy, and J. Ang, "QASMBench: A low-level quantum benchmark suite for NISQ evaluation and simulation," *ACM Trans. Quantum Comput.*, vol. 4, no. 2, 2023.
[6] T. Tomesh *et al.*, "SupermarQ: A scalable quantum benchmark suite," in *Proc. IEEE Int. Symp. High-Performance Comput. Archit. (HPCA)*, 2022.
[7] G. Li, Y. Ding, and Y. Xie, "Tackling the qubit mapping problem for NISQ-era quantum devices," in *Proc. ASPLOS*, 2019.
[8] P. Murali, J. M. Baker, A. Javadi-Abhari, F. T. Chong, and M. Martonosi, "Noise-adaptive compiler mappings for noisy intermediate-scale quantum computers," in *Proc. ASPLOS*, 2019.
[9] A. Javadi-Abhari *et al.*, "Quantum computing with Qiskit," arXiv:2405.08810, 2024.